

menler

Your turning point in the AI era.

AI APTITUDE QUESTION BANK

AI Networks and Infrastructure

About This Question Bank

A bank for engineers and technical operators who need to understand the infrastructure layer of AI — APIs, retrieval systems, deployment patterns, cost management, and security.

5

Sets

15

Questions per Set

75

Total Questions

SET GUIDE

Set 1 · APIs and Integration

APIs and integration — how to connect models to applications correctly

Set 2 · RAG and Vector Databases

RAG and vector databases — retrieval-augmented generation and semantic search

Set 3 · Deployment and Scaling

Deployment and scaling — taking AI systems from prototype to production

Set 4 · Cost and Latency

Cost and latency — optimising AI systems for speed and economics

Set 5 · Security and Access

Security and access — protecting AI systems and the data they handle

Each question has one correct answer. Select the best answer from the options provided.

Correct answers and explanations follow each question.

SET

1

APIs and Integration

Set 1 of 5 · 15 Questions · AI Networks and Infrastructure

Q1. What is an "API key" in the context of accessing an AI model?

- A. A configuration parameter that controls the model's output temperature.
- B. A credential that authenticates requests to an AI provider's API endpoints.**
- C. A licence that grants permission to use a specific model commercially.
- D. An encryption key used to secure the model's weights in transit.

Correct Answer: B**Explanation**

An API key is an authentication credential — a secret string that identifies the calling application to the AI provider. It is used to authenticate API requests, track usage, enforce rate limits, and bill correctly. It is not a configuration value, licence, or encryption mechanism.

Q2. What does a REST API return when a request succeeds?

- A. A binary model output that must be decoded before use.
- B. A confirmation message and the request ID for tracking purposes.
- C. The model's confidence score alongside the generated response.
- D. A response body (typically JSON) and an HTTP 200-level status code.**

Correct Answer: D**Explanation**

A successful REST API response includes an HTTP 200-level status code (200 OK, 201 Created, etc.) and a response body — for AI APIs, typically JSON containing the model's output. Binary encoding, confidence scores, and request-ID-only responses are not standard REST patterns.

Q3. What is the difference between synchronous and asynchronous API calls?

- A. Synchronous calls wait for the response; asynchronous calls return immediately and retrieve the result later.**
- B. Synchronous calls are encrypted; asynchronous calls are not.
- C. Synchronous calls are for small requests; asynchronous calls are for large ones.
- D. Synchronous calls use REST; asynchronous calls use WebSockets.

Correct Answer: A

Explanation

Synchronous calls block until the server responds. Asynchronous calls return immediately with a job ID; the caller polls for or receives a callback with the result when it is ready. Asynchronous patterns are essential for long-running AI tasks that would otherwise cause client timeouts.

Q4. What is "streaming" in the context of LLM API responses?

- A. Continuously processing an incoming audio or video feed through an AI model.
- B. Sending large documents to the AI API in small chunks to avoid size limits.
- C. Receiving tokens as they are generated rather than waiting for the full response.**
- D. A real-time learning mode where the model updates based on user feedback.

Correct Answer: C

Explanation

Streaming (server-sent events or WebSocket) delivers tokens to the client progressively as the model generates them, rather than waiting for the complete response. This dramatically improves perceived responsiveness — the user sees output appearing immediately rather than after a potentially long wait.

Q5. What HTTP status code indicates that an API request was rejected due to rate limiting?

- A. 503 Service Unavailable.
- B. 429 Too Many Requests.**
- C. 401 Unauthorized.
- D. 400 Bad Request.

Correct Answer: B

Explanation

429 Too Many Requests is the standard HTTP status code for rate limiting. It indicates the client has exceeded the allowed request frequency. 401 is for authentication failure; 400 is for malformed requests; 503 is for server-side unavailability.

Q6. What is "exponential backoff" and why is it used in AI API integrations?

- A. Retrying failed requests with increasing delays to avoid overwhelming the API.**
- B. A technique for gradually increasing batch size to optimise throughput.
- C. A method for reducing token usage by exponentially compressing prompts.
- D. A caching strategy that stores responses for exponentially increasing time periods.

Correct Answer: A

Explanation

Exponential backoff retries a failed API request after a delay, doubling the delay with each subsequent retry. This prevents retry storms from overwhelming a rate-limited or temporarily unavailable service, while still ensuring eventual success for transient failures.

Q7. What is a "system prompt" in the API context, and how does it differ from a user message?

- A. A system prompt is encrypted for security; a user message is sent in plain text.
- B. A system prompt is processed first by a faster model; user messages use the full model.
- C. A system prompt is visible to users; a user message is confidential.
- D. A system prompt sets persistent instructions for the model; a user message is the runtime input.**

Correct Answer: D

Explanation

In the API, a system prompt (role: "system") provides persistent context, persona, and constraints that apply to the entire conversation. User messages (role: "user") are the runtime inputs. The model treats these differently: system prompts carry higher instructional authority.

Q8. What does "max_tokens" control in an API request?

- A. The maximum number of tokens allowed in the user's input prompt.
- B. The total maximum tokens for the entire conversation context window.
- C. The maximum number of tokens the model will generate in its response.**
- D. The number of candidate responses the model generates before selecting the best.

Correct Answer: C

Explanation

max_tokens sets a ceiling on the response length. The model will stop generating once this limit is reached, potentially truncating the response. It does not limit input length or context size — those are controlled by the context window limit and separate input parameter validation.

Q9. What is the purpose of a "system fingerprint" or "model version pinning" in production API use?

- A. Ensuring a specific model version is used for reproducibility and preventing unexpected changes.**
- B. Verifying that the API response was generated by the claimed model and not tampered with.
- C. Tracking which system sent the API request for billing attribution.
- D. Ensuring the model uses the same random seed across all requests.

Correct Answer: A

Explanation

Model providers periodically update models under the same name. Version pinning ensures your application always runs on a specific, tested model version — preventing unexpected behaviour changes from silent model updates in production. This is critical for reproducible, testable production systems.

Q10. What is "context stuffing" and why is it problematic?

- A. A technique for fitting more data into a prompt by compressing or summarising it.
- B. An attack where malicious content is added to the context to manipulate the model.
- C. The process of loading model weights into GPU memory before inference.
- D. Filling the context window with excessive content, degrading model attention on key information.**

Correct Answer: D

Explanation

Context stuffing occurs when so much text is added to the prompt that the model's attention becomes diluted — relevant information is buried and the model may miss or deprioritise it. "Lost in the middle" research shows models attend better to information at the beginning and end of very long contexts.

Q11. What does an API "endpoint" refer to?

- A. The server hardware at the physical end of the network that runs the model.
- B. The specific URL that receives API requests for a particular functionality.**
- C. The final token the model generates before stopping.
- D. The response format returned at the end of a streaming response.

Correct Answer: B

Explanation

An API endpoint is the URL path that accepts requests for a specific function — e.g., `https://api.anthropic.com/v1/messages` is Anthropic's messages endpoint. Different endpoints provide different functionalities within the same API.

Q12. What is "prompt caching" and what problem does it solve?

- A. Storing common user prompts in a database to respond without calling the model.
- B. Caching model outputs so identical requests return the same response instantly.
- C. Reusing cached computation for repeated prompt prefixes to reduce cost and latency.**
- D. A technique for compressing prompts before sending them to the API.

Correct Answer: C

Explanation

Prompt caching (like Anthropic's prefix caching) allows the model to reuse the computed key-value representations of a repeated prompt prefix across multiple calls. This is highly valuable for applications that repeatedly use the same large system prompt or context — dramatically reducing cost and latency for the repeated portion.

Q13. What is the most important consideration when storing API keys in a production application?

- A. Keys must be rotated every 24 hours to prevent unauthorised use.
- B. Keys must be encrypted with AES-256 before being stored in any database.
- C. Keys must be different for development and production environments.
- D. Keys must be stored in environment variables or a secrets manager, never in code.**

Correct Answer: D

Explanation

API keys in source code are a critical security failure — they end up in version control, logs, and everywhere the code is deployed. The correct practice is environment variables or a dedicated secrets manager (AWS Secrets Manager, Vault, etc.), which keeps keys outside the codebase entirely.

Q14. What is "model routing" in a multi-model production system?

- A. Load-balancing requests across multiple instances of the same model.
- B. Directing different requests to different models based on task requirements and cost.**
- C. Routing model outputs to different downstream systems based on content type.
- D. Selecting the optimal serving hardware for each model inference request.

Correct Answer: B

Explanation

Model routing selects which model to use for each request based on factors like task complexity, required capability, cost constraints, and latency requirements. Simple queries route to cheaper, faster models; complex tasks to more capable ones. This optimises the cost-quality tradeoff across a production workload.

Q15. What does "API observability" mean in an AI production system?

- A. Monitoring request/response data, latency, errors, costs, and quality to understand system behaviour.**
- B. Making the AI API's internal workings transparent to end users.
- C. Ensuring the model's reasoning is explainable for each response it generates.
- D. Logging all user conversations for compliance and audit purposes.

Correct Answer: A

Explanation

API observability means instrumenting the system to capture the data needed to understand how it is performing: request volumes, error rates, latency distributions, token costs, and output quality metrics. Without observability, production issues are invisible until they cause user-facing failures.

SET

2

RAG and Vector Databases

Set 2 of 5 · 15 Questions · AI Networks and Infrastructure

Q16. What is "RAG" (Retrieval-Augmented Generation) and what problem does it solve?

- A. Training a model on domain-specific data to improve its in-domain performance.
- B. A technique for reducing hallucination by blocking uncertain model outputs.
- C. Supplementing a model with retrieved documents to answer questions it could not from training alone.**
- D. A method for compressing large datasets before feeding them to a language model.

Correct Answer: C**Explanation**

RAG retrieves relevant documents from an external knowledge base and includes them in the model's context. This allows the model to answer questions about information not in its training data (recent events, proprietary documents, domain-specific knowledge) without fine-tuning.

Q17. What is a "vector embedding" in the context of RAG?

- A. A numerical representation of text that captures semantic meaning in a high-dimensional space.**
- B. A compressed version of a document stored in a binary database format.
- C. A template that structures how retrieved documents are inserted into prompts.
- D. A hash of a document used to check for duplicates in the knowledge base.

Correct Answer: A**Explanation**

A vector embedding is a dense numerical vector (e.g., 1536 dimensions) produced by an embedding model that represents the semantic content of a text chunk. Semantically similar texts produce similar (nearby) vectors, enabling similarity-based retrieval.

Q18. How does a vector database enable semantic search?

- A. By indexing documents with keyword tags and returning exact keyword matches.
- B. By training a specialised model on the document corpus for each new query.
- C. By storing documents in a tree structure ordered by semantic topic.
- D. By comparing query embeddings to document embeddings using cosine or dot product similarity.**

Correct Answer: D

Explanation

A vector database stores document embeddings and enables approximate nearest-neighbour (ANN) search. When a query arrives, it is embedded by the same model, and the database returns documents whose embeddings are most similar to the query embedding — capturing semantic relevance, not just keyword overlap.

Q19. What is "chunking" in RAG and why does it matter?

- A. Compressing retrieved documents to reduce the tokens sent to the model.
- B. Splitting documents into smaller pieces that fit within retrieval and context limits.**
- C. Dividing the RAG workflow into modular processing stages.
- D. Grouping semantically similar documents together for batch embedding.

Correct Answer: B

Explanation

Documents must be split into chunks because embedding models and context windows have size limits. Chunk size is a critical design decision: too small and chunks lack context; too large and retrieval becomes imprecise (the relevant information is diluted across too much irrelevant content in the same chunk).

Q20. What is "hybrid search" in a RAG system?

- A. Running the same query through multiple embedding models and merging results.
- B. Searching both an internal database and the public internet for relevant documents.
- C. Combining vector similarity search with keyword (BM25) search to improve retrieval.**
- D. Using two different vector databases and selecting the better result for each query.

Correct Answer: C

Explanation

Hybrid search combines semantic vector search (which handles paraphrase and conceptual matching) with keyword-based BM25 search (which excels at exact term matching). Different queries benefit from different approaches, and combining them produces more reliable retrieval than either alone.

Q21. What is "re-ranking" in a RAG pipeline and when is it used?

- A. Running the model on retrieved documents to decide which ones are accurate.
- B. Reordering the model's generated output for better coherence and flow.
- C. A post-processing step that re-ranks model responses by quality score.
- D. A second-pass scoring step that reorders retrieved documents by relevance before passing to the model.**

Correct Answer: D

Explanation

After initial retrieval returns the top-K candidates by vector similarity, re-ranking applies a more expensive but more accurate relevance model (e.g., a cross-encoder) to reorder them. This improves the quality of the documents actually passed to the generation model.

Q22. What causes "retrieval failures" in a RAG system and how are they typically diagnosed?

- A. The retrieved documents do not contain the answer — diagnosed by checking retrieval results independently of generation.**
- B. The vector database is returning results too slowly, causing timeouts.
- C. The embedding model is mismatched with the vector database query format.
- D. The model is ignoring retrieved documents and generating from training knowledge.

Correct Answer: A

Explanation

Retrieval failure is when the pipeline retrieves irrelevant or incomplete documents, so the generation step has no basis for the correct answer. Diagnosing this requires evaluating retrieval independently — checking whether the correct documents are being retrieved for test queries, separate from generation quality.

Q23. What is "embedding model alignment" and why does it matter in RAG?

- A. The embedding model must be trained on the same data as the generation model.
- B. Documents and queries must be embedded by the same model for similarity comparisons to work.**
- C. Embedding dimensions must exactly match the vector database's storage format.
- D. The embedding model must be retrained whenever documents in the knowledge base change.

Correct Answer: B

Explanation

Embeddings are only comparable when generated by the same model — the vector space of Model A is not the same as Model B's. Querying a database of Model A embeddings with a Model B query embedding will return meaningless similarity scores. This alignment must be maintained across document ingestion and query time.

Q24. What is the most important metric to track for RAG retrieval quality?

- A. Mean retrieval latency — how quickly the vector database returns results.
- B. Embedding model perplexity on the document corpus.
- C. The cosine similarity score of the top-1 retrieved result for each query.
- D. Recall@K — whether the correct document is in the top-K retrieved results.**

Correct Answer: D

Explanation

Recall@K measures whether the relevant document is found within the top K retrieved results. This is the foundational retrieval metric — if the correct document is not in the retrieved set, generation cannot produce a correct answer regardless of how capable the generation model is.

Q25. What is "metadata filtering" in a vector database?

- A. Removing identifying information from documents before they are embedded.
- B. Filtering out low-quality embeddings based on their vector magnitude.
- C. Filtering retrieved documents by structured attributes (date, author, category) in addition to semantic similarity.**
- D. A security layer that prevents unauthorised users from querying certain documents.

Correct Answer: C

Explanation

Metadata filtering combines semantic similarity search with structured attribute filters — for example, "find semantically similar documents where date > 2024-01-01 and category = finance." This dramatically improves retrieval precision in applications where document scope matters.

Q26. What is "document freshness" a challenge for in RAG systems?

- A. Ensuring retrieved documents are readable and not too old to understand.
- B. Keeping the knowledge base up to date when source documents change or new ones are added.**
- C. Preventing the model from generating outdated information from its training data.
- D. Managing storage costs as the document corpus grows over time.

Correct Answer: B

Explanation

RAG knowledge bases must be actively maintained — when source documents are updated, deleted, or added, the index must be updated accordingly. Stale indices return outdated information. This is an ongoing operational requirement, not a one-time setup.

Q27. What is the "lost in the middle" problem in RAG with long contexts?

- A. Language models attend less to information in the middle of long contexts, missing retrieved content.**
- B. Long retrieved documents cause the model to lose track of the original user question.
- C. Documents retrieved in the middle of a session are processed less accurately than early ones.
- D. Chunking long documents causes the middle sections to be missing from the index.

Correct Answer: A

Explanation

"Lost in the middle" research shows that language models attend most strongly to the beginning and end of long contexts, with attention dropping for content in the middle. This means RAG systems that concatenate many retrieved documents may have the model miss or underweight the most relevant content if it lands in the middle.

Q28. What is the difference between "sparse" and "dense" retrieval?

- A. Sparse retrieval indexes a subset of documents; dense retrieval indexes all documents.
- B. Sparse retrieval is faster but less accurate; dense retrieval is slower but perfect.
- C. Sparse uses keyword matching (BM25); dense uses semantic embedding similarity.**
- D. Sparse retrieval works for short queries; dense retrieval handles multi-sentence queries.

Correct Answer: C

Explanation

Sparse retrieval (BM25, TF-IDF) represents documents as weighted keyword vectors — mostly zeros. Dense retrieval uses neural embedding vectors — all dimensions have values. Sparse excels at exact term matching; dense excels at semantic and paraphrase matching. Hybrid systems combine both.

Q29. What is "answer grounding" in RAG and why is it important?

- A. Physically grounding the RAG system to a secure server infrastructure.
- B. Verifying that retrieved documents are from trusted, authoritative sources.
- C. Ensuring the model's answer uses the exact wording from retrieved documents.
- D. Ensuring the model's answer is based on and citable from the retrieved documents.**

Correct Answer: D

Explanation

Answer grounding means the model's response should be traceable to specific retrieved content — not generated from training knowledge or hallucinated. Grounded answers can be verified by checking the source documents; ungrounded answers cannot. Grounding is the primary mechanism by which RAG reduces hallucination.

Q30. What is the most important operational consideration for a production RAG system?

- A. Embedding model inference speed to minimise query-time latency.
- B. Document ingestion pipeline reliability — ensuring the index stays fresh and complete.**
- C. Vector database storage cost as the document corpus grows.
- D. The number of documents returned per retrieval call (top-K parameter).

Correct Answer: B

Explanation

The most important operational requirement is a reliable, continuous ingestion pipeline that keeps the index current with the source documents. A RAG system with an out-of-date index provides wrong or missing information. Everything else — speed, cost, K parameter — is secondary to index freshness and completeness.

SET

3

Deployment and Scaling

Set 3 of 5 · 15 Questions · AI Networks and Infrastructure

Q31. What is "time to first token" (TTFT) and why does it matter for user experience?

- A. The latency before the first token appears — drives perceived responsiveness in streaming UIs.**
- B. The time required for the model to read and process the first token of input.
- C. The delay between model deployment and the first successful API request.
- D. The time it takes for the first token in a prompt to be embedded and stored.

Correct Answer: A

Explanation

TTFT is the time from request submission to the first character appearing in the response. In streaming UIs, low TTFT makes responses feel immediate even if total generation takes longer. High TTFT makes the interface feel slow even if total response time is reasonable.

Q32. What is "tokens per second" (TPS) and what does it measure in LLM serving?

- A. The rate at which input tokens are processed by the model before generation begins.
- B. The throughput of the vector database during retrieval operations.
- C. The rate at which the model generates output tokens — higher means faster text generation.**
- D. The API's capacity for processing concurrent streaming requests.

Correct Answer: C

Explanation

TPS (tokens per second) measures output generation speed — how quickly the model produces response tokens after the first token appears. It determines how fast text streams to the user and how many requests the system can serve in parallel at a given quality level.

Q33. What is a "GPU" and why is it essential for LLM inference?

- A. A specialised memory chip that stores model weights for fast retrieval during inference.
- B. A processor with thousands of cores optimised for the parallel matrix operations in neural network inference.**
- C. A hardware accelerator that translates model outputs into binary for network transmission.
- D. A co-processor that handles tokenisation and embedding before the main model runs.

Correct Answer: B

Explanation

GPUs (Graphics Processing Units) have thousands of small cores optimised for parallel floating-point operations — exactly what neural network inference (matrix multiplication, attention computation) requires. CPUs have far fewer cores and cannot perform these operations at the scale needed for LLM inference.

Q34. What is "model sharding" in the context of LLM deployment?

- A. Splitting user requests across multiple independent model instances for load balancing.
- B. Breaking model responses into shards that are assembled client-side.
- C. A compression technique that reduces model size by removing redundant weights.
- D. Distributing model weights across multiple GPUs because the model is too large for one device.**

Correct Answer: D

Explanation

Model sharding (tensor or pipeline parallelism) partitions model layers or weight tensors across multiple GPUs to handle models that exceed single-device memory. Without sharding, 70B+ parameter models cannot be deployed at all on consumer or cloud GPU hardware.

Q35. What is "serving latency percentile" (e.g., p99) and why does it matter?

- A. The latency that 99% of requests complete within — captures worst-case user experience.**
- B. The average latency for the top 99% most common request types.
- C. A benchmark score measuring the model's inference speed on standard hardware.
- D. The percentage of requests that complete within a predefined latency SLA.

Correct Answer: A

Explanation

The p99 latency is the value that 99% of requests complete within — meaning 1% of requests are slower. The p99 matters more than average latency for user experience because it captures the tail behaviour: occasional slow requests that users experience as timeouts or hangs.

Q36. What is "cold start" in serverless AI deployment?

- A. An inference error caused by the model receiving an input it was not trained on.
- B. The latency spike from initialising a new container or loading model weights on first request.**
- C. A deployment strategy that starts with a small model and scales to larger models under load.
- D. The process of pre-warming the GPU cache before inference begins.

Correct Answer: B

Explanation

Cold start latency occurs when a serverless function or container that has been idle must initialise — loading model weights into GPU memory can take 10-60+ seconds for large models. This creates severe first-request latency spikes. Mitigation strategies include keep-warm instances, weight pre-caching, and smaller model deployment.

Q37. What is "autoscaling" in an AI serving infrastructure?

- A. Automatically adjusting the model's generation parameters based on query complexity.
- B. A technique for dynamically resizing batch sizes based on GPU memory availability.
- C. Automatically updating model weights when new training data becomes available.
- D. Automatically adding or removing compute instances based on request load.**

Correct Answer: D

Explanation

Autoscaling monitors request queue depth, latency, or CPU/GPU utilisation and adds new serving instances when demand increases, removing them when demand drops. This allows the system to handle traffic spikes without over-provisioning expensive GPU resources during low-traffic periods.

Q38. What is the primary tradeoff between model size and serving cost?

- A. Larger models have lower latency but higher memory requirements.
- B. Larger models are cheaper to run because they require fewer API calls.
- C. Larger models cost more per token but may produce higher quality output.**
- D. Larger models produce more tokens per request, increasing total cost disproportionately.

Correct Answer: C

Explanation

Larger models require more GPU memory, more computation per token, and thus cost more per inference call. The tradeoff is quality versus cost: a 7B model costs less to serve than a 70B model, but may produce lower quality outputs for complex tasks. Optimal model selection minimises cost for each task's quality requirement.

Q39. What is "inference batching" and how does it improve GPU utilisation?

- A. Breaking a single large request into multiple smaller batches for parallel processing.
- B. Processing multiple requests simultaneously in a single GPU forward pass.**
- C. Pre-generating common responses in advance for instant retrieval.
- D. Grouping similar prompts together to improve cache hit rates.

Correct Answer: B

Explanation

GPU utilisation is often low when processing individual requests because the GPU's parallel processing capacity is underutilised. Batching multiple requests together into a single forward pass makes better use of the GPU's parallelism, improving throughput (requests/second) at the cost of some additional latency per request.

Q40. What is "vLLM" and what problem does it solve?

- A. A high-throughput serving engine that improves GPU memory efficiency using PagedAttention.**
- B. A validation library for verifying that LLM outputs meet quality thresholds.
- C. A vector database optimised for large-scale language model embedding storage.
- D. A load balancer designed specifically for distributing LLM API requests.

Correct Answer: A

Explanation

vLLM uses PagedAttention to manage KV cache memory in blocks, similar to how operating systems manage virtual memory. This dramatically reduces memory waste (fragmentation) in the KV cache, allowing more concurrent requests on the same GPU hardware and significantly improving serving throughput.

Q41. What is "SLA" (Service Level Agreement) in the context of AI API deployment?

- A. A legal agreement governing the permitted uses of an AI model in production.
- B. A security standard that API providers must meet for enterprise deployment.
- C. A committed performance guarantee defining uptime, latency, and throughput thresholds.**
- D. A model accuracy guarantee specifying minimum benchmark performance.

Correct Answer: C

Explanation

An SLA defines measurable service commitments: uptime percentage (e.g., 99.9%), latency targets (e.g., p99 < 2s), throughput minimums, and error rate ceilings. SLAs are the contractual basis for enterprise AI deployments and determine compensation when commitments are missed.

Q42. What is "horizontal scaling" in AI serving infrastructure?

- A. Increasing the hardware specification of existing serving instances.
- B. Extending the model's context window to handle longer inputs.
- C. Scaling the model's parameter count to improve output quality.
- D. Adding more serving instances in parallel to increase total capacity.**

Correct Answer: D

Explanation

Horizontal scaling (scale out) adds more serving instances, distributing the load across them. This is contrasted with vertical scaling (scale up), which increases the resources of existing instances. Horizontal scaling is generally more cost-effective and flexible for AI serving as it allows granular capacity adjustments.

Q43. What is the most important thing to include in an AI service health dashboard?

- A. GPU utilisation and memory consumption per serving instance.
- B. Error rate, latency percentiles, and request volume — the three pillars of service health.**
- C. Cost per request and total monthly API spend.
- D. Model accuracy scores from the latest evaluation run.

Correct Answer: B

Explanation

Error rate (are requests succeeding?), latency percentiles (are requests fast enough?), and request volume (is demand normal?) are the three core service health signals. GPU utilisation, costs, and accuracy scores are important secondary metrics but are not the primary health indicators.

Q44. What is the "deployment canary" strategy in AI model updates?

- A. Deploying the new model to a test environment and canary testing for safety issues.
- B. Using a smaller, cheaper model as a canary to catch harmful requests before the main model.
- C. Routing a small percentage of traffic to the new model version before full rollout.**
- D. A gradual rollout that increases model context window size incrementally.

Correct Answer: C

Explanation

Canary deployment routes a small fraction of live traffic (e.g., 5%) to the new model version while the rest continues on the old version. This allows real-world performance comparison and catches issues before they affect all users, enabling rollback if the new version underperforms.

Q45. What is "model deprecation" and what should production systems do about it?

- A. When a model version is retired by the provider — systems should pin to specific versions and have migration plans.**
- B. When a model's accuracy degrades over time due to distribution shift in production inputs.
- C. The process of removing an underperforming model from a multi-model routing system.
- D. A fine-tuning technique that reduces model size by removing deprecated capabilities.

Correct Answer: A

Explanation

Model deprecation occurs when an AI provider retires a specific model version, forcing users to migrate. Production systems should pin to specific versions, monitor deprecation announcements, and have tested migration plans ready before deprecated versions are shut down to avoid unplanned outages.

SET

4

Cost and Latency

Set 4 of 5 · 15 Questions · AI Networks and Infrastructure

Q46. What is the primary cost driver for LLM API usage?

- A. The number of API requests made regardless of their content.
- B. The time spent waiting for the model to complete generation.
- C. The size of the model accessed, independent of token count.
- D. The number of input and output tokens processed per request.**

Correct Answer: D**Explanation**

LLM API pricing is almost universally based on token count — input tokens and output tokens priced separately (output tokens are typically more expensive). The number of API calls matters for rate limiting but not for cost; time is not directly billed; model size affects the per-token rate, not an independent billing dimension.

Q47. What is "token efficiency" and why does it matter at scale?

- A. Ensuring the model generates the maximum useful information per token output.
- B. Achieving the required output quality with the fewest possible tokens — directly reduces cost.**
- C. A measure of how accurately the model represents information in each token.
- D. The ratio of useful content to total content in the model's training data.

Correct Answer: B**Explanation**

Token efficiency means achieving the required quality with fewer tokens. At scale, a 30% reduction in average prompt length translates directly to a 30% reduction in input token costs — compounding significantly across millions of requests. Prompt engineering for efficiency is a direct cost lever.

Q48. Which approach most effectively reduces output token costs in a production system?

- A. Using a higher temperature to generate more focused responses.
- B. Switching to a smaller model that generates shorter responses.
- C. Instructing the model to be concise and specifying exactly the output format needed.**
- D. Increasing the max_tokens limit to allow the model to be more complete.

Correct Answer: C

Explanation

Instructing the model to be concise with a specific output format (e.g., "respond in exactly 3 bullet points") directly reduces output token count without degrading quality for the use case. Higher temperature increases randomness; smaller models may reduce quality; higher max_tokens increases potential cost.

Q49. What is "cost per query" in AI production systems and how is it calculated?

- A. Total API cost divided by number of queries — the primary unit economics metric.**
- B. The infrastructure cost of running one GPU for the duration of one query.
- C. The product of latency and compute cost per second for each inference call.
- D. The cost of the embedding model call plus the generation model call per RAG query.

Correct Answer: A

Explanation

$\text{Cost per query} = \text{total API spend} / \text{number of queries}$. It is the core unit economics metric for AI product profitability. It captures the full cost of serving each user request, enabling comparison to revenue per query to determine whether the AI feature is economically viable.

Q50. A RAG system makes 3 API calls per user query: embedding, retrieval, and generation. The most expensive component is typically:

- A. Embedding — dense embedding models are computationally the most expensive component.
- B. Retrieval — vector database queries at scale dominate infrastructure costs.
- C. All three are approximately equal in cost for a well-designed system.
- D. Generation — output tokens from large models cost significantly more than embedding.**

Correct Answer: D

Explanation

Generation using a large language model is by far the most expensive component. Embedding models are small and inexpensive. Vector database queries are generally cheap at typical scales. The generation step — especially if it involves long outputs from a frontier model — typically accounts for 90%+ of the API cost.

Q51. What is "caching" in the context of LLM production costs?

- A. Pre-loading model weights into GPU memory to reduce cold start latency.
- B. Using compression to reduce the size of API responses in transit.
- C. Storing responses for identical or near-identical queries to avoid repeated API calls.**
- D. Storing user conversation history locally to reduce API calls for context reconstruction.

Correct Answer: C

Explanation

Response caching stores model outputs for queries that have been answered before. When the same (or similar) query arrives, the cached response is returned instantly without an API call. For applications with repetitive queries (FAQs, common scenarios), caching can reduce API costs dramatically.

Q52. What is the "price-performance" consideration when selecting a model for a production use case?

- A. Using the cheapest model that reliably achieves the required quality threshold.**
- B. Selecting the highest-performance model and optimising for cost later.
- C. Matching the model size exactly to the complexity of the task at hand.
- D. Using the same model for all tasks to simplify infrastructure management.

Correct Answer: A

Explanation

The correct approach is to define the minimum quality threshold the use case requires, then find the cheapest model that reliably meets it. Over-provisioning (using an expensive model where a cheaper one would suffice) wastes money; under-provisioning (using a cheap model that doesn't meet quality requirements) destroys user value.

Q53. What is "token budget management" in a complex AI workflow?

- A. Allocating a maximum number of tokens to each user account per month.
- B. Tracking and controlling token usage across all steps to prevent cost overruns.**
- C. Setting the max_tokens parameter to match the expected response length.
- D. Compressing prompts using a tokeniser to minimise input token count.

Correct Answer: B

Explanation

Token budget management tracks cumulative token usage across a multi-step workflow, ensuring costs do not escalate unexpectedly. A workflow that chains 10 steps might spend 5x more tokens than anticipated if each step's context grows. Active budget tracking and enforcement prevents cost surprises.

Q54. What is the most cost-effective approach for handling a task that requires only simple text formatting?

- A. Use the largest model available to ensure perfect output quality.
- B. Use a deterministic rule-based system and avoid calling an AI model entirely.
- C. Use a small, fast model — simple tasks do not justify frontier model costs.**
- D. Use the frontier model with a very low max_tokens to limit cost.

Correct Answer: C

Explanation

Simple formatting tasks (standardising capitalisation, converting between formats, extracting fixed patterns) are best handled by small, fast, cheap models or even rule-based systems. Using frontier models for tasks they massively overkill wastes budget that could be spent on tasks that genuinely need capability.

Q55. What is "inference cost optimisation" at the infrastructure level?

- A. Reducing the number of model layers to speed up inference computation.
- B. Optimising the model's sampling parameters to generate fewer tokens.
- C. Minimising the size of training data to reduce the cost of future fine-tuning runs.
- D. Maximising GPU utilisation through batching, quantisation, and efficient scheduling.**

Correct Answer: D

Explanation

At the infrastructure level, cost optimisation focuses on maximising the useful work done per GPU dollar: batching requests together for parallel processing, quantising weights to fit more models per device, and scheduling to keep GPUs busy. These are engineering levers distinct from prompt-level or model-selection levers.

Q56. What is "usage-based pricing" vs "flat-rate pricing" for AI APIs?

- A. Usage-based charges per token; flat-rate charges a fixed fee regardless of consumption.**
- B. Usage-based charges per API call; flat-rate charges per model version accessed.
- C. Usage-based scales down with volume; flat-rate charges the same regardless of scale.
- D. Usage-based is for inference; flat-rate is for training and fine-tuning.

Correct Answer: A

Explanation

Usage-based pricing (the standard for most AI APIs) charges based on actual token consumption. Flat-rate pricing charges a fixed subscription fee regardless of usage — better for predictable high-volume workloads. The right choice depends on usage patterns, volume, and whether predictable costs or pay-per-use economics are preferable.

Q57. What is the primary risk of optimising for cost before establishing baseline quality?

- A. Cost optimisations are harder to implement once the system is in production.
- B. You may optimise for the wrong objective if quality requirements are not yet defined.**
- C. Cheap models are less secure and introduce additional safety risks.
- D. Cost optimisation requires retraining the model which is expensive.

Correct Answer: B

Explanation

Optimising for cost before quality is defined is premature optimisation. If the quality bar is not established, you do not know what "good enough" looks like, and cost cuts may degrade quality below acceptable thresholds. Always establish quality baselines first, then optimise cost within those constraints.

Q58. What is "latency vs throughput" tradeoff in AI serving?

- A. Higher throughput models produce lower latency responses by generating faster.
- B. Latency is a user experience concern; throughput is an infrastructure concern.
- C. Low latency and high throughput are always in conflict and cannot both be achieved.
- D. Low latency often requires dedicated resources that reduce utilisation; high throughput benefits from batching that increases latency.**

Correct Answer: D

Explanation

Batching requests together improves throughput (requests/second) by maximising GPU utilisation, but increases per-request latency (requests must wait for the batch to fill). Dedicated single-request serving minimises latency but lowers throughput. This fundamental tradeoff must be resolved for each use case based on whether latency or throughput is the binding constraint.

Q59. What is "compute cost normalisation" when comparing AI pricing across providers?

- A. Converting costs to a per-token or per-1M-token basis for apples-to-apples comparison.**
- B. Adjusting for hardware differences between cloud providers when benchmarking.
- C. Normalising model output quality scores before comparing cost-effectiveness.
- D. Converting provider costs to a common currency for international cost comparisons.

Correct Answer: A

Explanation

Providers use different pricing units (per token, per 1K tokens, per 1M tokens). Normalising all prices to per-1M-token makes comparisons valid. Without normalisation, a "cheap" provider may simply be reporting in a different unit, making direct comparison misleading.

Q60. What is the biggest risk of using a free tier AI API in a production system?

- A. Free tiers use less capable model versions than paid tiers.
- B. Free tier usage cannot be monitored or tracked for debugging.
- C. Rate limits and reliability SLAs are not appropriate for production workloads.**
- D. Free tier API keys expire unpredictably, causing production outages.

Correct Answer: C

Explanation

Free tiers are designed for development and experimentation. They have severe rate limits, no reliability SLAs, and no support commitments. Using a free tier in production means rate limits will be hit under normal load, and there is no contractual guarantee of uptime or support when things go wrong.

SET

5

Security and Access

Set 5 of 5 · 15 Questions · AI Networks and Infrastructure

Q61. What is "data privacy" the most critical concern for in AI API integrations?

- A. Ensuring sensitive data sent to AI providers is not stored, used for training, or exposed.**
- B. Ensuring the AI model does not generate personally identifiable information in outputs.
- C. Preventing users from extracting the AI provider's proprietary model weights.
- D. Encrypting all API communications between the client and the model provider.

Correct Answer: A**Explanation**

When sensitive data (PII, confidential business information, health records) is sent to AI providers in API requests, there are critical questions about how it is stored, whether it is used for training, who can access it, and whether it is subject to data residency requirements. These are the primary privacy concerns for AI API integrations.

Q62. What is "output filtering" in a production AI system?

- A. Limiting the length of model outputs to prevent context window overflow.
- B. Filtering input prompts before they are sent to the model.
- C. Selecting only the highest-quality outputs from multiple model generations.
- D. Post-processing model outputs to detect and block harmful or inappropriate content.**

Correct Answer: D**Explanation**

Output filtering applies safety classifiers or rule-based checks to model outputs before they are delivered to users, catching harmful content that slipped through the model's own safety training. It is a defence-in-depth layer, not a substitute for model-level safety.

Q63. What is "zero data retention" (ZDR) in AI API contracts and when is it required?

- A. A technical setting that prevents the model from caching previous responses.
- B. A contractual guarantee that the provider does not store API request/response data.**
- C. A compliance mode where the model refuses to generate any data about users.
- D. A policy that prevents training data from being retained after a model is released.

Correct Answer: B

Explanation

Zero data retention is an enterprise contract provision where the AI provider guarantees that API request and response data is not logged, stored, or used for any purpose after processing. It is required for handling highly sensitive data (healthcare, legal, financial) under regulations like HIPAA or contractual obligations.

Q64. What is "access control" most important for in a multi-user AI application?

- A. Preventing users from accessing the AI system during maintenance windows.
- B. Limiting the number of API calls each user can make per day.
- C. Ensuring users can only query with data and permissions appropriate to their role.**
- D. Ensuring the AI model only generates content appropriate for each user's age group.

Correct Answer: C

Explanation

In multi-user AI applications (enterprise SaaS, internal tools), access control ensures each user's AI interactions only touch data they are authorised to see. Without this, a RAG system might retrieve confidential documents belonging to another user, or an agent might access resources outside its permission scope.

Q65. What is "audit logging" in AI governance and what must it capture?

- A. A tamper-evident log of who queried the AI, what they asked, and what it responded.**
- B. A technical log of API errors and latency metrics for system debugging.
- C. A training data provenance record showing where model knowledge came from.
- D. A compliance certificate demonstrating the model meets regulatory standards.

Correct Answer: A

Explanation

AI audit logs for governance must capture who made the request (identity), what was requested (input), what the AI responded (output), and when (timestamp). This record is essential for investigating incidents, demonstrating compliance, and detecting misuse — it must be tamper-evident and retained per policy.

Q66. What is "model security" concerned with in an AI deployment context?

- A. Ensuring the AI model's weights are free of security vulnerabilities.
- B. Verifying that the AI model was trained on legally compliant data.
- C. Preventing unauthorised access, model extraction, and adversarial manipulation of the AI system.**
- D. Protecting the AI model from being overwhelmed by high volumes of requests.

Correct Answer: C

Explanation

Model security covers: preventing unauthorised API access, protecting against model extraction attacks (reverse-engineering the model through queries), defending against adversarial inputs designed to manipulate model behaviour, and securing the serving infrastructure. These are the attack surfaces specific to deployed AI systems.

Q67. What is "PII detection" and why is it important in AI pipelines?

- A. Detecting when AI outputs contain names and personal details that should be anonymised.
- B. A compliance check that verifies model training data was PII-free.
- C. An output filter that removes all names from AI-generated text.
- D. Identifying personally identifiable information in data before it is sent to AI models.**

Correct Answer: D

Explanation

PII detection scans data before it enters AI pipelines to identify personally identifiable information (names, emails, SSNs, medical records). If PII is detected, it can be masked, anonymised, or flagged before being sent to an external AI provider, preventing inadvertent privacy violations.

Q68. What is the most important governance requirement for AI use in a regulated industry?

- A. Using only AI models that have received regulatory certification for the specific industry.
- B. Documented policies for AI use, output verification, and human oversight of consequential decisions.**
- C. Ensuring AI systems are built on open-source models for transparency.
- D. Limiting AI use to tasks that have been individually approved by a compliance committee.

Correct Answer: B

Explanation

Regulated industry AI governance requires documented policies (what AI is used for, by whom, under what constraints), verification procedures (how AI outputs are checked before use), and human oversight for decisions with regulatory consequences. Certification, open-source requirements, and committee approval are supporting elements, not the foundation.

Q69. What is "rate limiting" from an API provider's perspective?

- A. Limiting the length of model outputs to manage server resource consumption.
- B. Throttling the speed of model generation to improve response consistency.
- C. Restricting the number of API requests or tokens per time period per client to ensure fair access.**
- D. Restricting which model versions a particular client tier can access.

Correct Answer: C

Explanation

Rate limiting protects the shared API infrastructure from overconsumption by any single client. It is typically enforced per API key, per account tier, with limits on requests per minute, tokens per minute, and sometimes tokens per day. Rate limiting ensures equitable access and prevents accidental or intentional resource exhaustion.

Q70. What is "vendor lock-in" as a risk in AI infrastructure decisions?

- A. Becoming so dependent on one AI provider's proprietary features that switching is prohibitively costly.**
- B. A legal liability when using AI provider data in ways that violate their terms.
- C. An infrastructure risk when a single provider hosts all production AI workloads.
- D. A quality risk when a provider's model quality deteriorates after you have built on it.

Correct Answer: A

Explanation

Vendor lock-in occurs when a system is built heavily around provider-specific APIs, proprietary formats, or unique features that do not have equivalents elsewhere. Switching providers then requires significant re-engineering. This becomes a business risk if the provider raises prices, degrades quality, or discontinues a service.

Q71. What is the correct approach to handling model provider outages in a production AI system?

- A. Queue all requests and retry them once the provider recovers.
- B. Fallback to an alternative provider or degrade gracefully to non-AI functionality.**
- C. Alert users that the AI service is temporarily unavailable and ask them to wait.
- D. Switch to a cached version of the model running locally on application servers.

Correct Answer: B

Explanation

Production AI systems must handle provider outages gracefully: routing to an alternative provider (multi-vendor strategy), falling back to rule-based or non-AI alternatives for critical functions, or queuing non-urgent requests. Simply showing an error or asking users to wait is unacceptable for business-critical applications.

Q72. What is "infrastructure as code" (IaC) important for in AI deployment?

- A. Writing the AI model's system prompt in a code-based configuration format.
- B. Using code to generate training data for fine-tuning AI models.
- C. Defining the AI model's architecture parameters in configuration files.
- D. Reproducible, version-controlled, auditable deployment of AI serving infrastructure.**

Correct Answer: D

Explanation

IaC (Terraform, CloudFormation, etc.) defines serving infrastructure in version-controlled code. This ensures deployments are reproducible, changes are auditable, and rollbacks are fast. For AI systems where security and compliance matter, IaC provides the governance trail required for infrastructure changes.

Q73. What is the most important security practice for AI API keys in a team environment?

- A. Sharing a master API key only with senior team members via secure messaging.
- B. Rotating all API keys every 24 hours to prevent long-term exposure.
- C. Using per-service, per-environment keys with least-privilege scope and rotation policies.**
- D. Storing API keys in a shared encrypted spreadsheet accessible to the team.

Correct Answer: C

Explanation

Per-service, per-environment keys with least-privilege scope mean each service has its own key with only the permissions it needs. This limits the blast radius of a compromised key, enables granular revocation, and provides clear attribution in audit logs. Shared master keys and spreadsheet storage are significant security anti-patterns.

Q74. What is "content moderation" as an infrastructure concern in AI deployments?

- A. Manually reviewing AI-generated content before it is published to users.
- B. Automatically screening both inputs and outputs for harmful content at scale.**
- C. Configuring the AI model's safety settings to block certain content categories.
- D. Limiting which users can access AI features based on age or verification status.

Correct Answer: B

Explanation

Content moderation as infrastructure means building systematic, automated checks — input scanners for harmful prompts, output classifiers for unsafe content — that operate at the scale of production traffic. This is a complementary layer to model-level safety, not a substitute for it, operating at the serving layer.

Q75. What does "AI governance" mean at the organisational level?

- A. The technical controls that prevent AI models from taking unauthorised actions.
- B. Compliance with specific AI regulations in the organisation's operating jurisdiction.
- C. A committee that approves each new AI model deployment before it goes to production.
- D. Policies, processes, and oversight structures for responsible AI deployment and use.**

Correct Answer: D

Explanation

AI governance is the organisational framework — policies defining permitted uses, risk assessment processes, oversight mechanisms, accountability structures, incident response procedures — that ensures AI is used responsibly across the organisation. Technical controls, regulatory compliance, and approval committees are all elements within this broader framework.



Your turning point in the AI era.

END OF QUESTION BANK

*Your turning
point starts
here.*